SMART CHEF & PANTRY AI

JSON Request / Response & Error Models

Versiyon: 1.0
Backend: Python / Flask
Frontend: Flutter / Dart
Format: JSON
Authentication: Bearer JWT

---

1. Amaç

Bu dokümanın amacı Flutter mobil uygulaması ile Flask REST API arasında gönderilecek ve alınacak veri modellerini standartlaştırmaktır.

Bu doküman sayesinde:

- Flutter model sınıfları,
- Flask request validation,
- Flask response schema'ları,
- Repository katmanı,
- API error handling,
- Authentication response'ları

aynı veri sözleşmesine göre geliştirilecektir.

Temel prensip:

Flutter Model
↕
JSON
↕
Flask Schema
↕
Service
↕
Database Model

Flutter ile PostgreSQL arasında doğrudan model eşleşmesi yapılmayacaktır.

---

2. Genel JSON Kuralları

API'de:

- JSON property isimleri `snake_case` kullanılacaktır.
- UUID değerleri string olarak gönderilecektir.
- Tarih/saat değerleri ISO 8601 formatında olacaktır.
- Boş değerler gerektiğinde `null` olarak gönderilecektir.
- Boolean değerleri `true/false` olacaktır.
- Quantity değerleri sayısal olarak tutulacaktır.
- Kullanıcıya gösterilecek özel miktar ifadeleri için `display_quantity` kullanılacaktır.

Örneğin:

```
{
  "quantity": 0.5,
  "unit": "kg",
  "display_quantity": "yarım kg"
}
```

---

3. Standard API Response

Başarılı tekil veri:

```
{
  "success": true,
  "data": {},
  "message": "Operation successful."
}
```

Liste:

```
{
  "success": true,
  "data": {
    "items": [],
    "pagination": {}
  }
}
```

Hata:

```
{
  "success": false,
  "error": {
    "code": "ERROR_CODE",
    "message": "Human readable message.",
    "details": {}
  }
}
```

`details` alanı zorunlu değildir.

---

4. Authentication Models

4.1 SocialLoginRequest

Flutter'ın backend'e göndereceği model:

```
{
  "provider": "google",
  "id_token": "GOOGLE_ID_TOKEN"
}
```

Alanlar

Alan	Tip	Zorunlu
provider	String	Evet
id_token	String	Evet

Provider

MVP:

google

İleride:

apple
facebook

gibi provider'lar eklenebilir.

---

5. SocialLoginResponse

Başarılı login:


```
{
  "success": true,
  "data": {
    "access_token": "JWT_ACCESS_TOKEN",
    "token_type": "Bearer",
    "user": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Esma Gelebek",
      "email": "user@example.com",
      "profile_image_url": "https://example.com/profile.jpg"
    }
  },
  "message": "Login successful."
}
```


---

6. User Model

Flutter tarafında karşılığı:

UserModel

JSON:


```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Esma Gelebek",
  "email": "user@example.com",
  "profile_image_url": "https://example.com/profile.jpg"
}
```


Alanlar:

Alan	Tip	Nullable

id	String	Hayır
name	String	Hayır
email	String	Hayır
profile_image_url	String	Evet

---

7. Ingredient Models

7.1 IngredientResponse

```
{
  "id": "ingredient-uuid",
  "name": "Domates",
  "category": "Vegetable"
}
```

Flutter modeli:

IngredientModel

Alanlar:

Alan	Tip
id	String
name	String
category	String?

---

8. Ingredient List Response

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": "ingredient-1",
        "name": "Domates",
```

```
      "category": "Vegetable"
    },
    {
      "id": "ingredient-2",
      "name": "Patates",
      "category": "Vegetable"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 2
  }
 }
}
```


---

9. Pagination Model

Tüm liste endpoint'lerinde gerektiğinde ortak pagination modeli kullanılacaktır.


```
{
  "page": 1,
  "limit": 20,
  "total": 100,
  "total_pages": 5
}
```


Flutter:

PaginationModel

Alanlar:

Alan	Tip
page	int
limit	int
total	int
total_pages	int

---

10. Recipe Model

Temel recipe modeli:

```
{
  "id": "recipe-uuid",
  "title": "Domatesli Tavuk",
  "description": "Pratik bir tavuk tarifi.",
  "image_url": "https://example.com/image.jpg",
  "instructions": "1. Tavuğu doğrayın...",
  "source_url": "https://example.com/recipe",
  "youtube_url": "https://youtube.com/watch?v=123",
  "category": "Main Course"
}
```

Flutter:

RecipeModel

Alanlar:

Alan	Tip	Nullable
id	String	Hayır
title	String	Hayır
description	String	Evet
image_url	String	Evet
instructions	String	Hayır
source_url	String	Evet
youtube_url	String	Evet
category	String	Evet

---

11. Recipe Ingredient Model

Recipe içerisindeki ingredient:

```
{
  "id": "ingredient-uuid",
  "name": "Domates",
  "quantity": 2,
  "unit": "adet",
  "display_quantity": "2 adet",
  "is_optional": false
}
```


Flutter:

RecipeIngredientModel

Alanlar:

Alan	Tip	Nullable
id	String	Hayır
name	String	Hayır
quantity	double	Evet
unit	String	Evet
display_quantity	String	Evet
is_optional	bool	Hayır

---

12. Recipe Detail Response


```
{
  "success": true,
  "data": {
    "id": "recipe-uuid",
    "title": "Domatesli Tavuk",
    "description": "Pratik bir tavuk tarifi.",
    "image_url": "https://example.com/image.jpg",
    "instructions": "1. Tavuğu doğrayın...",
    "source_url": "https://example.com/recipe",
    "youtube_url": "https://youtube.com/watch?v=123",
    "category": "Main Course",
    "ingredients": [
      {
        "id": "ingredient-1",
```

```
      "name": "Tavuk",
      "quantity": 500,
      "unit": "g",
      "display_quantity": "500 g",
      "is_optional": false
    },
    {
      "id": "ingredient-2",
      "name": "Domates",
      "quantity": 2,
      "unit": "adet",
      "display_quantity": "2 adet",
      "is_optional": false
    }
  ]
 }
}
```


---

13. Recipe Recommendation Request

Bu model recommendation sisteminin giriş verisidir.


```
{
 "ingredient_ids": [
   "ingredient-1",
   "ingredient-2",
   "ingredient-3"
 ]
}
```


Flutter:

RecipeRecommendationRequest

Model:

ingredient_ids: List<String>

Validation

Aşağıdaki durumlar kabul edilmemelidir:

ingredient_ids = null
ingredient_ids = []
ingredient_ids = ["invalid-id"]

En az bir ingredient gönderilmesi gerekmektedir.

---

14. Matched Ingredient Model

Recipe recommendation sonucu kullanıcıda bulunan ingredient'ları belirtmek için:

```
{
  "id": "ingredient-1",
  "name": "Domates"
}
```

Flutter:

MatchedIngredientModel

---

15. Missing Ingredient Model

Kullanıcının sahip olmadığı ingredient:

```
{
  "id": "ingredient-4",
  "name": "Biber"
}
```

Flutter:

MissingIngredientModel

İleride quantity bilgisi de eklenebilir:

```
{
  "id": "ingredient-4",
  "name": "Biber",
  "quantity": 2,
```

```
  "unit": "adet"
}
```

---

16. Recipe Recommendation Item

Recommendation sonucunda kullanılacak model:

```
{
  "id": "recipe-uuid",
  "title": "Domatesli Tavuk",
  "image_url": "https://example.com/image.jpg",
  "match_percentage": 75,
  "matched_ingredients": [
    {
      "id": "ingredient-1",
      "name": "Tavuk"
    },
    {
      "id": "ingredient-2",
      "name": "Domates"
    }
  ],
  "missing_ingredients": [
    {
      "id": "ingredient-3",
      "name": "Biber"
    }
  ]
}
```

Flutter:

RecipeRecommendationItem

---

17. Recipe Recommendation Response

```
{
  "success": true,
  "data": {
    "recipes": [
```

```
    {
      "id": "recipe-1",
      "title": "Domatesli Tavuk",
      "image_url": "https://example.com/image.jpg",
      "match_percentage": 75,
      "matched_ingredients": [
        {
          "id": "ingredient-1",
          "name": "Tavuk"
        },
        {
          "id": "ingredient-2",
          "name": "Domates"
        }
      ],
      "missing_ingredients": [
        {
          "id": "ingredient-3",
          "name": "Biber"
        }
      ]
    }
  ]
 }
}
```


---

18. Match Percentage Validation

`match_percentage`:

0 ≤ match_percentage ≤ 100

Tam eşleşme:


```
{
 "match_percentage": 100
}
```


Hiç eşleşme:


```
{
 "match_percentage": 0
}
```

Backend hesaplama sonucunu Flutter tekrar hesaplamayacaktır.

Bu özellikle önemlidir.

Backend → hesaplar
Flutter → gösterir

Böylece business logic iki farklı yerde tutulmaz.

---

19. Favorite Models

19.1 Add Favorite Request

```
{
  "recipe_id": "recipe-uuid"
}
```

Flutter:

AddFavoriteRequest

19.2 Favorite Response

```
{
  "id": "favorite-uuid",
  "recipe_id": "recipe-uuid",
  "created_at": "2026-09-27T10:00:00Z"
}
```

---

20. Favorite List Item

```
{
  "id": "favorite-uuid",
  "recipe": {
    "id": "recipe-uuid",
    "title": "Domatesli Tavuk",
```

```
    "image_url": "https://example.com/image.jpg"
  },
  "created_at": "2026-09-27T10:00:00Z"
}
```


Flutter:

FavoriteModel

---

21. Cart Models

21.1 Add Cart Item Request


```
{
  "ingredient_id": "ingredient-uuid",
  "quantity": 2,
  "unit": "adet"
}
```


Flutter:

AddCartItemRequest

---

22. Cart Item Model


```
{
  "id": "cart-item-uuid",
  "ingredient": {
    "id": "ingredient-uuid",
    "name": "Domates"
  },
  "quantity": 5,
  "unit": "adet"
}
```


Flutter:

CartItemModel

---

23. Update Cart Item Request

```
{
  "quantity": 7
}
```

Flutter:

UpdateCartItemRequest

Quantity:

quantity > 0

olmalıdır.

---

24. Cart Response

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": "cart-item-1",
        "ingredient": {
          "id": "ingredient-1",
          "name": "Domates"
        },
        "quantity": 5,
        "unit": "adet"
      }
    ],
    "total_items": 1
  }
}
```

---

25. Error Model

Bütün API hataları ortak bir format kullanacaktır.

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid request data.",
    "details": {}
  }
}
```

Flutter:

ApiErrorModel

---

26. Error Codes

Authentication

AUTH_REQUIRED
INVALID_TOKEN
TOKEN_EXPIRED
SOCIAL_AUTH_FAILED
USER_NOT_FOUND

Validation

VALIDATION_ERROR
INVALID_UUID
INVALID_QUANTITY
EMPTY_INGREDIENT_LIST
INVALID_PROVIDER

Recipe

RECIPE_NOT_FOUND
RECIPE_INACTIVE
NO_RECIPES_FOUND

Ingredient

INGREDIENT_NOT_FOUND
INGREDIENT_ALREADY_EXISTS

Favorite

FAVORITE_NOT_FOUND
FAVORITE_ALREADY_EXISTS

Cart

CART_ITEM_NOT_FOUND
INVALID_CART_QUANTITY

Authorization

FORBIDDEN
ADMIN_REQUIRED

Server

INTERNAL_SERVER_ERROR
DATABASE_ERROR

---

27. Validation Error Örneği

Kullanıcı quantity olarak `0` gönderirse:


```
{
  "success": false,
  "error": {
    "code": "INVALID_QUANTITY",
    "message": "Quantity must be greater than zero.",
    "details": {
      "field": "quantity",
      "value": 0
    }
```

```
  }
}
```

---

28. Authentication Error Örneği

JWT gönderilmemişse:

```
{
  "success": false,
  "error": {
    "code": "AUTH_REQUIRED",
    "message": "Authentication is required."
  }
}
```

---

29. Not Found Error Örneği

Recipe bulunamadığında:

```
{
  "success": false,
  "error": {
    "code": "RECIPE_NOT_FOUND",
    "message": "Recipe not found."
  }
}
```

---

30. Conflict Error Örneği

Kullanıcı aynı recipe'yi tekrar favorilere eklerse:

```
{
  "success": false,
  "error": {
    "code": "FAVORITE_ALREADY_EXISTS",
    "message": "Recipe is already in favorites."
  }
```

}

---

31. Flutter Model Yapısı

Flutter'da API modellerinin önerilen organizasyonu:

```
lib/
└── features/
    ├── ingredients/
    │   └── data/
    │       └── models/
    │           ├── ingredient_model.dart
    │           └── ingredient_response_model.dart
    │
    ├── recipes/
    │   └── data/
    │       └── models/
    │           ├── recipe_model.dart
    │           ├── recipe_ingredient_model.dart
    │           ├── recipe_recommendation_item.dart
    │           └── recipe_recommendation_request.dart
    │
    ├── favorites/
    │   └── data/
    │       └── models/
    │           └── favorite_model.dart
    │
    ├── cart/
    │   └── data/
    │       └── models/
    │           ├── cart_item_model.dart
    │           └── cart_response_model.dart
    │
    └── auth/
        └── data/
            └── models/
                ├── user_model.dart
                ├── social_login_request.dart
                └── social_login_response.dart
```

---

32. Flask Schema Yapısı

Flask tarafında request validation için:

```
app/
└── schemas/
    ├── auth.py
    ├── ingredient.py
    ├── recipe.py
    ├── favorite.py
    ├── cart.py
    └── common.py
```

kullanılacaktır.

Örneğin:

SocialLoginSchema
RecipeRecommendationSchema
AddFavoriteSchema
AddCartItemSchema
UpdateCartItemSchema

---

33. API Model ile Database Model Ayrımı

Önemli tasarım kararı:

Database Model ≠ API Model

Örneğin database:

```
Recipe
├── id
├── title
├── description
├── instructions
├── source_url
├── youtube_url
├── created_at
└── updated_at
```

API:

```
RecipeResponse
├── id
```

├── title
├── description
├── image_url
├── instructions
├── source_url
├── youtube_url
├── category
└── ingredients

Database'e ait teknik alanların tamamı API'ye açılmayacaktır.

Bu yaklaşım ileride database yapısı değişse bile API sözleşmesinin mümkün olduğunca stabil kalmasını sağlar.

---

34. Nullability Kuralları

Frontend tarafında nullable alanlar açıkça tanımlanacaktır.

Örneğin:

String? imageUrl
String? description
String? sourceUrl
String? youtubeUrl
String? category

Ancak:

String id
String title
String instructions

nullable olmayacaktır.

---

35. DateTime Formatı

Tarih alanları:

ISO 8601

formatında gönderilecektir.

Örnek:

2026-09-27T10:00:00Z

Flutter:

DateTime.parse(...)

ile parse edebilir.

---

36. API Contract'ın Temel Kuralı

Flutter ve Flask geliştirilirken aşağıdaki prensip korunacaktır:

Önce Contract
↓
Sonra Model
↓
Sonra Service
↓
Sonra UI

Yani Flutter'da bir ekran geliştirilirken API response'un nasıl görüneceği sonradan tahmin edilmeyecektir.

Önce bu dokümandaki contract kullanılacaktır.

---

37. MVP'de Kullanılacak Temel Model Listesi

Authentication

UserModel
SocialLoginRequest
SocialLoginResponse

Ingredients

IngredientModel
IngredientListResponse

Recipes

RecipeModel
RecipeIngredientModel
RecipeRecommendationRequest
RecipeRecommendationItem
RecipeRecommendationResponse
MatchedIngredientModel
MissingIngredientModel

Favorites

AddFavoriteRequest
FavoriteModel

Cart

AddCartItemRequest
UpdateCartItemRequest
CartItemModel
CartResponse

Common

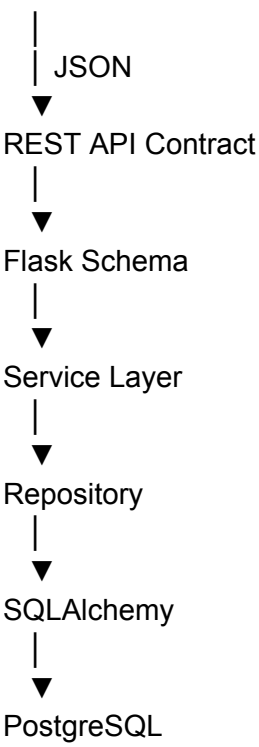
PaginationModel
ApiErrorModel
ApiResponse

---

38. Sonuç

Bu doküman ile Smart Chef & Pantry AI'ın Flutter ve Flask tarafları arasında kullanılacak temel veri sözleşmesi tanımlanmıştır.

Mimari olarak:



şeklinde ilerlenmesi planlanmaktadır.

Bir sonraki aşamada bu JSON modellerinin Flutter ve Flask tarafında hangi sınıflara dönüşeceği ve katmanların birbirinden nasıl ayrılacağı tanımlanacaktır.